

CS224N Assignment 2: Written Solutions

Diego Alcaraz

1. Word2Vec

(a) Naive-Softmax Loss and Cross-Entropy

The skip-gram Word2Vec model aims to learn the conditional probability distribution

$$P(O = o \mid C = c)$$

of an outside word o given a center word c . This probability is modeled using the softmax function:

$$P(O = o \mid C = c) = \hat{y}_o = \frac{\exp(u_o^\top v_c)}{\sum_{w \in \mathcal{V}} \exp(u_w^\top v_c)}$$

The naive-softmax loss for a single pair (c, o) is:

$$J_{\text{naive-softmax}}(v_c, o, U) = -\log P(O = o \mid C = c)$$

We can view this loss as the cross-entropy between the true distribution y and the predicted distribution $\hat{y}$. Here, y is a one-hot vector such that $y_o = 1$ and $y_w = 0$ for all $w \neq o$.

The cross-entropy loss is:

$$H(y, \hat{y}) = -\sum_{w \in \mathcal{V}} y_w \log \hat{y}_w$$

Since y is one-hot, the summation reduces to:

$$H(y, \hat{y}) = -\log \hat{y}_o$$

Thus, the naive-softmax loss is exactly the cross-entropy loss between the true and predicted distributions.

(b) Gradient with Respect to the Center Vector

(i) Gradient Derivation

The loss can be written as:

$$J = -u_o^\top v_c + \log \sum_{w \in \mathcal{V}} \exp(u_w^\top v_c)$$

Taking the gradient with respect to v_c :

$$\frac{\partial J}{\partial v_c} = -u_o + \sum_{w \in \mathcal{V}} \hat{y}_w u_w$$

In vectorized form:

$$\boxed{\frac{\partial J}{\partial v_c} = U(\hat{y} - y)}$$

(ii) When Is the Gradient Zero?

The gradient is zero when:

$$\hat{y} = y$$

That is, when the predicted distribution exactly matches the true one-hot distribution. At this point, the model assigns full probability to the correct outside word, and no further update is required.

(iii) Interpretation of the Gradient Terms

The gradient consists of two terms:

$$U(\hat{y} - y) = U\hat{y} - Uy$$

When performing gradient descent:

$$v_c \leftarrow v_c - \alpha(U\hat{y} - Uy)$$

- The term $Uy = u_o$ pulls the center vector v_c toward the true outside word vector, strengthening the correct semantic relationship.
- The term $U\hat{y}$ pushes v_c away from all predicted outside word vectors, weighted by their predicted probabilities, reducing incorrect associations.

Overall, this update moves v_c closer to the true context word and away from incorrect ones.

(c) Effect of L2 Normalization

Consider two word vectors u_x and u_y such that:

$$u_x = \alpha u_y, \quad \alpha > 0$$

After L2 normalization:

$$\frac{u_x}{\|u_x\|_2} = \frac{u_y}{\|u_y\|_2}$$

When normalization removes useful information: If the magnitude of word vectors encodes information such as sentiment strength or word importance, normalization removes this signal. For example, words like “good” and “excellent” may differ primarily in magnitude, which would be lost after normalization.

When normalization does not remove useful information: If downstream tasks rely only on vector direction (e.g., cosine similarity), normalization preserves the relevant semantic relationships.

2. Machine Learning & Neural Networks

(a) Adam Optimizer

(i) Momentum

Adam keeps an exponentially decaying average of past gradients:

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) \nabla J(\theta_t)$$

Momentum reduces variance in gradient updates by smoothing noisy minibatch gradients. This prevents oscillations in steep directions and allows the optimizer to move more consistently in directions of persistent descent, improving convergence speed and stability.

(ii) Adaptive Learning Rates

Adam also tracks the second moment:

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2)(\nabla J(\theta_t))^2$$

The update rule is:

$$\theta_{t+1} = \theta_t - \alpha \frac{m_{t+1}}{\sqrt{v_{t+1}}}$$

Parameters with smaller historical gradients receive larger updates, while those with larger gradients receive smaller updates. This balances learning across parameters and improves optimization in deep networks.

(b) Dropout

(i) Scaling Factor

Dropout is defined as:

$$h_{\text{drop}} = \gamma d \odot h$$

where each element of d is 1 with probability $(1 - p_{\text{drop}})$.

Taking expectation:

$$\mathbb{E}[h_{\text{drop},i}] = \gamma(1 - p_{\text{drop}})h_i$$

Setting this equal to h_i yields:

$$\gamma = \frac{1}{1 - p_{\text{drop}}}$$

(ii) Training vs Evaluation

Dropout is applied during training to prevent feature co-adaptation and encourage robust representations. During evaluation, dropout is not applied because we want deterministic predictions using the full network capacity.

3. Dependency Parsing

Sentence:

I presented my findings at the NLP conference.

We use a transition-based dependency parser with SHIFT, LEFT-ARC, and RIGHT-ARC operations.

Parsing Transitions

Stack	Buffer	New Dependence
[ROOT]	[I, presented, my, findings, at, the, NLP, conference]	–
[ROOT, I]	[presented, my, findings, at, the, NLP, conference]	–
[ROOT, I, presented]	[my, findings, at, the, NLP, conference]	–
[ROOT, presented]	[my, findings, at, the, NLP, conference]	presented $\rightarrow$ I

[ROOT, presented, my]	[findings, at, the, NLP, conference]	–
[ROOT, presented, my, findings]	[at, the, NLP, conference]	–
[ROOT, presented, findings]	[at, the, NLP, conference]	findings → my
[ROOT, presented]	[at, the, NLP, conference]	presented → fin
[ROOT, presented, at]	[the, NLP, conference]	–
[ROOT, presented, at, the]	[NLP, conference]	–
[ROOT, presented, at, the, NLP]	[conference]	–
[ROOT, presented, at, the, conference]	[]	conference → N
[ROOT, presented, at, conference]	[]	conference → th
[ROOT, presented, conference]	[]	conference → at
[ROOT, presented]	[]	presented → co
[ROOT]	[]	ROOT → prese

4. Attention Masks

The attention mask assigns $-\infty$ to attention scores corresponding to padding tokens before applying the softmax. This ensures that padded positions receive zero probability after softmax and do not contribute to the attention output.

This masking is necessary because padding tokens are artificial and contain no semantic meaning. Without masking, attention would incorrectly distribute probability mass to padding positions, resulting in distorted context representations.